ANO
2025



ATIVIDADE PRÁTICA
ESTRUTURA DE DADOS I

VINICIUS GONÇALVES. RU: 4471599

Prof. Me. Bruno Kostiuk

*Apresentação de **Código da Questão**:*

node.c

```
#include <stdio.h>
#include <stdlib.h>
#include "node.h"

void insertWithoutPriority(struct Node *head, struct Node *newNode)
{
    struct Node *currentNode = head;
    int count = 1;

    while (currentNode->next != NULL)
    {
        currentNode = currentNode->next;
    }

    if (currentNode->priority == 'C') {
        count = currentNode->number + 1;
    }

    newNode->number = count;
    currentNode->next = newNode;
    newNode->next = NULL;
}
```

```
void insertWithPriority(struct Node *head, struct Node *newNode)
{
    struct Node *currentNode = head;
    int count = 301;

    while(currentNode->next != NULL && currentNode->next->priority == 'P')
    {
        currentNode = currentNode->next;
        count++;
    }

    struct Node *next = currentNode->next;
    newNode->number = count;
    newNode->next = next;
    currentNode->next = newNode;
}
```

node.h

```
struct Node {
    int number;
    char priority;
    struct Node *next;
};

void insertWithoutPriority(struct Node *head, struct Node *newNode);
void insertWithPriority(struct Node *head, struct Node *newNode);
```

main.c

```
#include <stdio.h>
```

```
#include <stdlib.h>
#include "node.h"

struct Node* createNode(char priority) {
    struct Node* newNode = (struct Node*) malloc(sizeof(struct Node));
    newNode->number = 0;
    newNode->priority = priority;
    newNode->next = NULL;
    return newNode;
}

void serveCustomer(struct Node **head) {
    if (*head == NULL || (*head)->next == NULL) {
        printf("Empty queue!\n");
        return;
    }

    struct Node *first = (*head)->next;
    printf("Calling card customer %d (%c) for service\n", first->number, first->
priority);

    (*head)->next = first->next;
    free(first);
}

void printClientList(struct Node *head)
{
    struct Node *currentNode = head->next;
    if (currentNode == NULL) {
        printf("Empty queue.\n");
        return;
```

```
    }
    while (currentNode != NULL) {
        printf("%d (%c)\n", currentNode->number, currentNode->priority);
        currentNode = currentNode->next;
    }
}

void menu(struct Node *clientList) {
    int option;
    char type;
    do {
        printf("\nMenu:\n");
        printf("1 Add client to queue\n");
        printf("2 Show queue\n");
        printf("3 Call client\n");
        printf("4 Exit\n");
        printf("Choose an option: ");
        scanf("%d", &option);

        switch(option) {
            case 1:
                printf("Enter the card type (C or P): ");
                scanf(" %c", &type);
                if (type == 'C') {
                    insertWithoutPriority(clientList, createNode('C'));
                } else if (type == 'P') {
                    insertWithPriority(clientList, createNode('P'));
                } else {
                    printf("Invalid type.\n");
                }
```

```
            break;
        case 2:
            printClientList(clientList);
            break;
        case 3:
            serveCustomer(&clientList);
            break;
        case 4:
            printf("Finishing.\n");
            break;
        default:
            printf("Invalid option. Try again.\n");
      }
   } while (option != 4);
}

int main()
{
   struct Node* clientList = (struct Node*) malloc(sizeof(struct Node));
   clientList->number = -1;
   clientList->priority = ' ';
   clientList->next = NULL;

   menu(clientList);

   return 0;
}
```

*Apresentação de **Saída do Console da Questão**:*

```
Menu:
1 Add client to queue
2 Show queue
3 Call client
4 Exit
Choose an option: 1
Enter the card type (C or P): C

Menu:
1 Add client to queue
2 Show queue
3 Call client
4 Exit
Choose an option: 1
Enter the card type (C or P): C

Menu:
1 Add client to queue
2 Show queue
3 Call client
4 Exit
Choose an option: 1
Enter the card type (C or P): C

Menu:
1 Add client to queue
2 Show queue
3 Call client
4 Exit
Choose an option: 1
Enter the card type (C or P): P

Menu:
1 Add client to queue
2 Show queue
3 Call client
4 Exit
Choose an option: 1
Enter the card type (C or P): P
```

```
Menu:
1 Add client to queue
2 Show queue
3 Call client
4 Exit
Choose an option: 1
Enter the card type (C or P): C

Menu:
1 Add client to queue
2 Show queue
3 Call client
4 Exit
Choose an option: 1
Enter the card type (C or P): C

Menu:
1 Add client to queue
2 Show queue
3 Call client
4 Exit
Choose an option: 1
Enter the card type (C or P): P

Menu:
1 Add client to queue
2 Show queue
3 Call client
4 Exit
Choose an option: 1
Enter the card type (C or P): P

Menu:
1 Add client to queue
2 Show queue
3 Call client
4 Exit
Choose an option: 1
Enter the card type (C or P): P
```

```
Menu:
1 Add client to queue
2 Show queue
3 Call client
4 Exit
Choose an option: 2
301 (P)
302 (P)
303 (P)
304 (P)
305 (P)
1 (C)
2 (C)
3 (C)
4 (C)
5 (C)
```

```
Menu:
1 Add client to queue
2 Show queue
3 Call client
4 Exit
Choose an option: 3
Calling card customer 301 (P) for service

Menu:
1 Add client to queue
2 Show queue
3 Call client
4 Exit
Choose an option: 3
Calling card customer 302 (P) for service

Menu:
1 Add client to queue
2 Show queue
3 Call client
4 Exit
Choose an option: 2
303 (P)
304 (P)
305 (P)
1 (C)
2 (C)
3 (C)
4 (C)
5 (C)
```